

MODULE 1

Foundations of Agentic AI

Understanding Autonomous Intelligence Systems

Session Roadmap



Introduction to Agentic AI

Definitions, distinctions, characteristics, and why now



Core Architecture of an AI Agent

Five-layer architecture, execution loop, ReAct pattern



Types of Agents

Reactive, deliberative, autonomous, multi-agent systems



Real-world Applications

Copilots, research assistants, workflow automation

SECTION 1

Introduction to Agentic AI

Defining what makes AI systems truly autonomous

What is an Agentic System?

Agentic AI is an artificial intelligence system that can accomplish a **specific goal** with limited supervision.....*IBM*



Perceives

Senses inputs from
users & environment



Reasons

Plans steps and
decides strategy



Acts

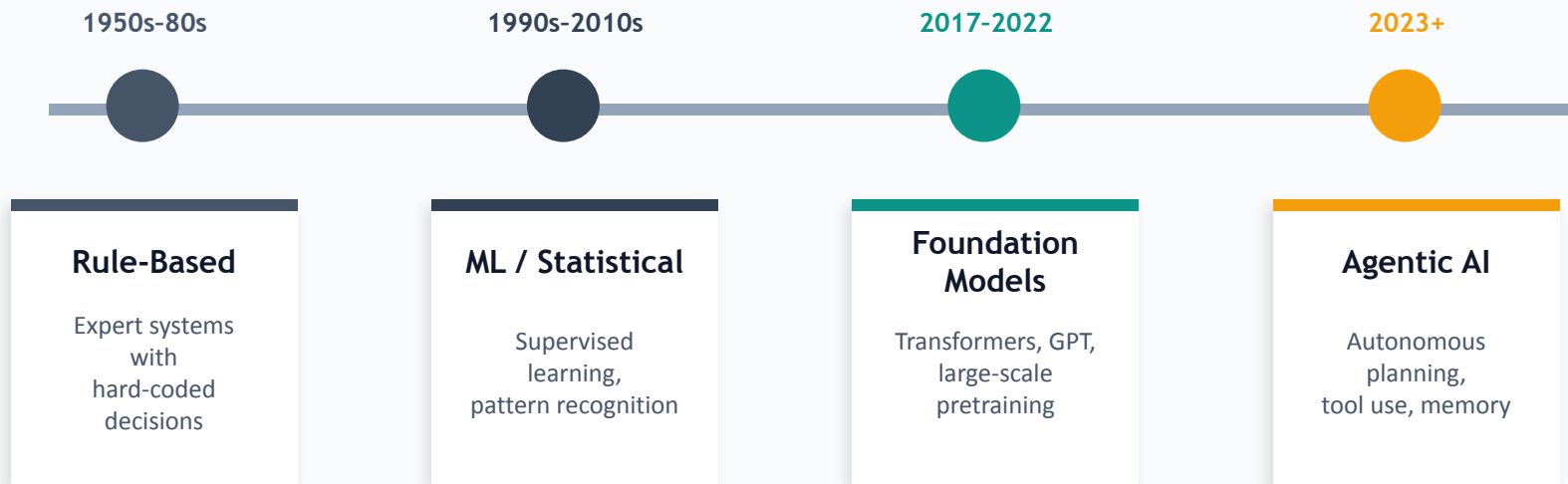
Executes via tools,
APIs, code



Iterates

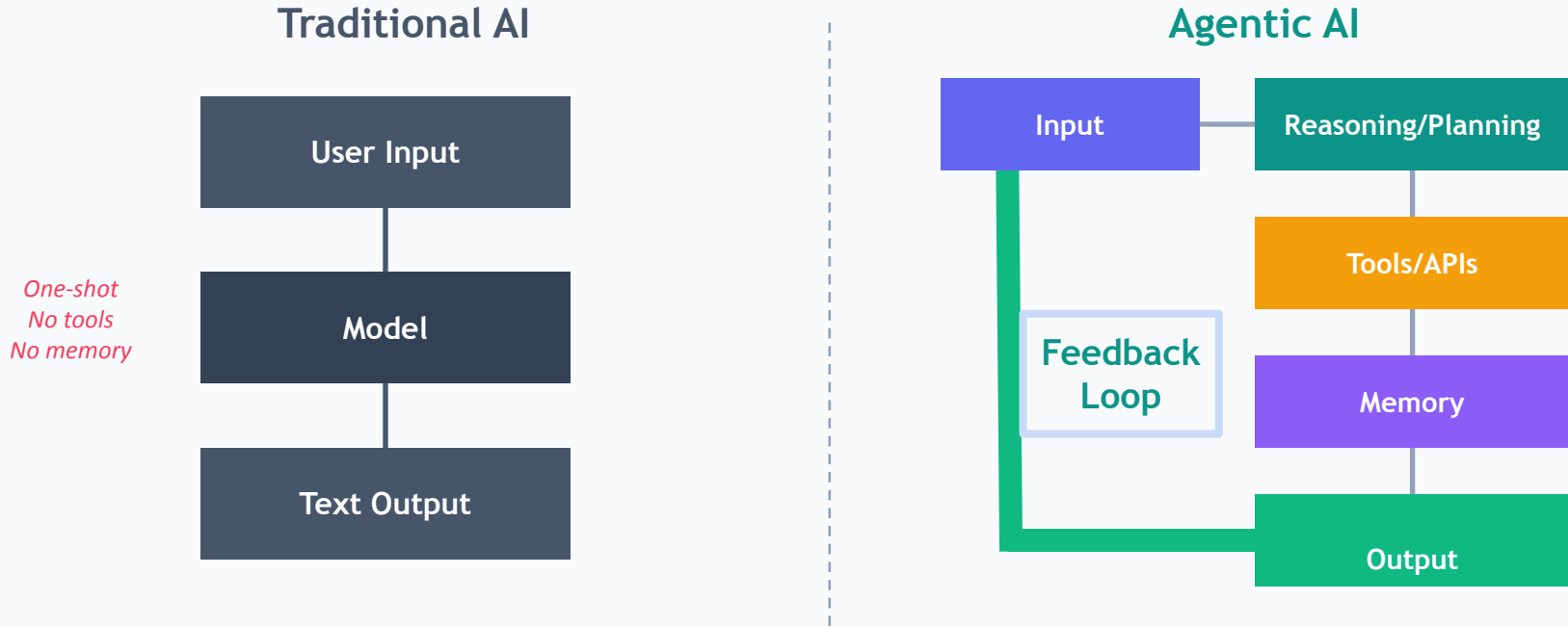
Evaluates results
and loops again

The Evolution Toward Agentic AI



The convergence of LLM reasoning, tool-use APIs, and orchestration frameworks made agentic AI practical starting in 2023.

System Diagram: Traditional vs Agentic



Traditional AI is a single-turn, no tool use, no persistent memory. Agentic AI is a loop that reasons, acts, evaluates, and repeats until the goal is achieved.

What is a Chatbot?

A chatbot is a conversational interface that responds to user messages, typically operating in a request-response pattern, with limited planning, memory, and tool use.

Turn-Based

Processes one message at a time in isolation

Stateless

Limited or session-based memory

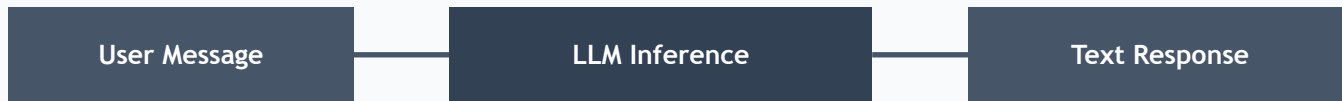
Passive

Only responds when directly prompted

Limited or no external tool use

Cannot call APIs, run code, or search

Chatbot System Flow:



What is an Autonomous Agent?

An autonomous agent independently pursues goals by observing, planning, executing actions via tools, and adapting, maintaining memory across steps and sessions.

Multi-Step

Chains actions to achieve complex goals

Stateful

Maintains context across steps and sessions

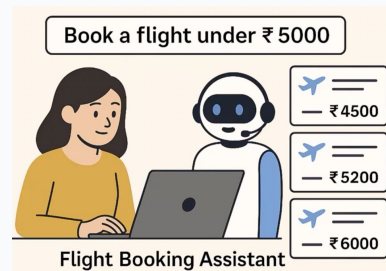
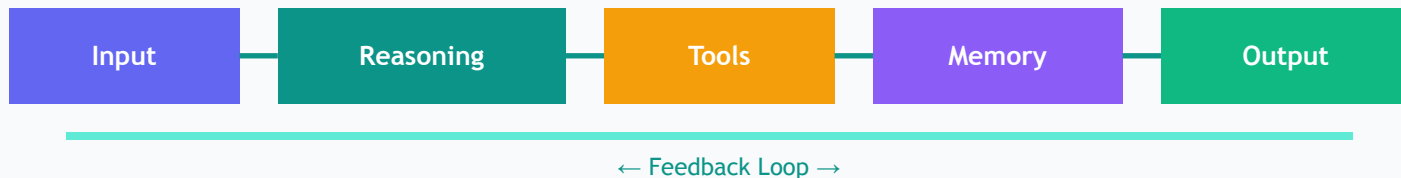
Proactive

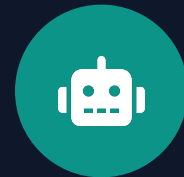
Initiates actions and makes decisions alone

Tool-Using

Calls APIs, runs code, searches the web

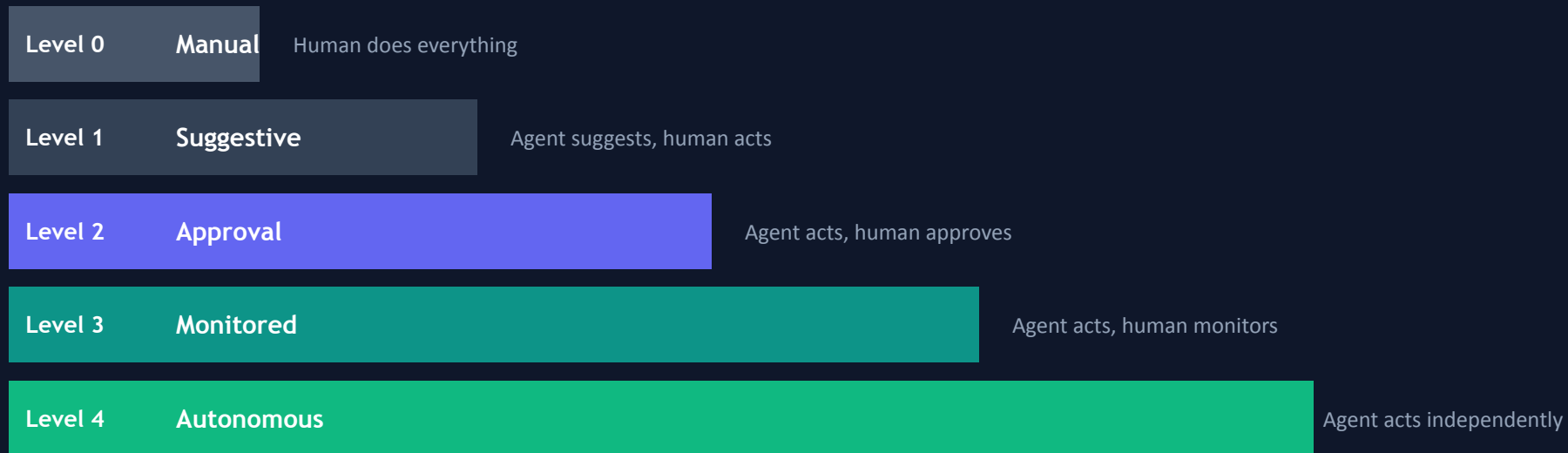
Agent System Flow:





Autonomy

The ability to operate independently, making decisions, selecting tools, and executing actions without requiring human approval at every step.



The Autonomy Spectrum



Where Do Production Systems Sit Today?

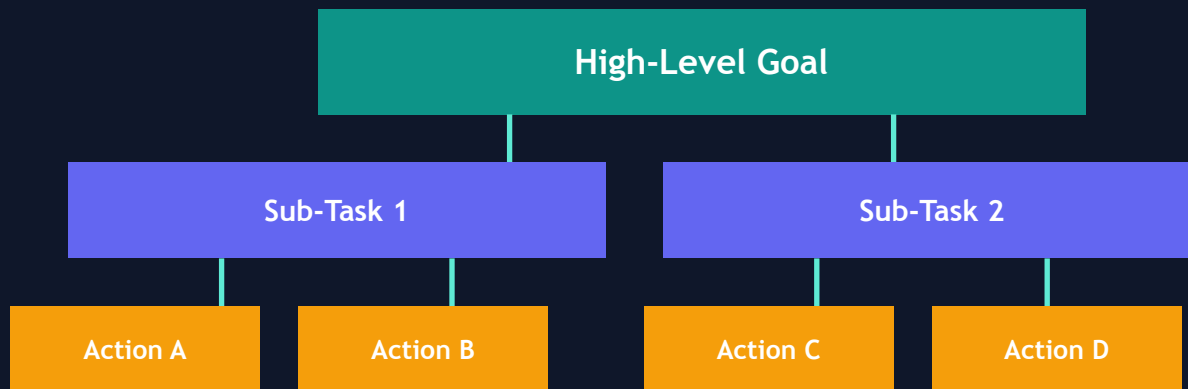
Most enterprise deployments are in the Copilot to Assistive Agent range. Fully autonomous agents are emerging in well-scoped domains (code generation, data analysis, customer support triage) where errors are recoverable and the action space is bounded.

Key insight: The right autonomy level depends on risk profile. High-stakes domains need more human oversight; low-stakes domains can tolerate higher autonomy.



Goal-Orientation

Agents pursue objectives. Every action moves toward a defined goal, and progress is continuously evaluated against that goal.

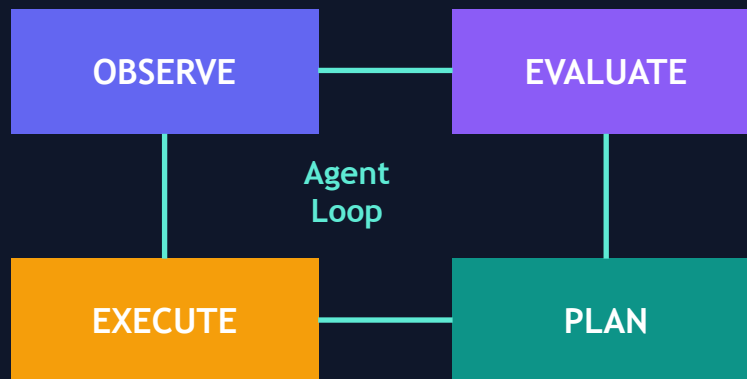


Goal Decomposition Tree



Iterative Decision-Making

Agents operate in loops: observe, evaluate, plan, execute. This feedback loop is the fundamental distinction from one-shot systems.



Why Now? Enabling Technologies



LLM Reasoning

GPT-4, Claude, Gemini can plan, reason, decompose tasks, and generate structured tool calls



Function Calling / Tool Use

Native API support for structured tool invocation, models output JSON with tool name + params



Vector Databases & RAG

Pinecone, Weaviate, ChromaDB enable long-term memory via embedding-based retrieval



Orchestration Frameworks

LangChain, LangGraph, CrewAI, AutoGen provide agent scaffolding and workflow management



Infrastructure Maturity

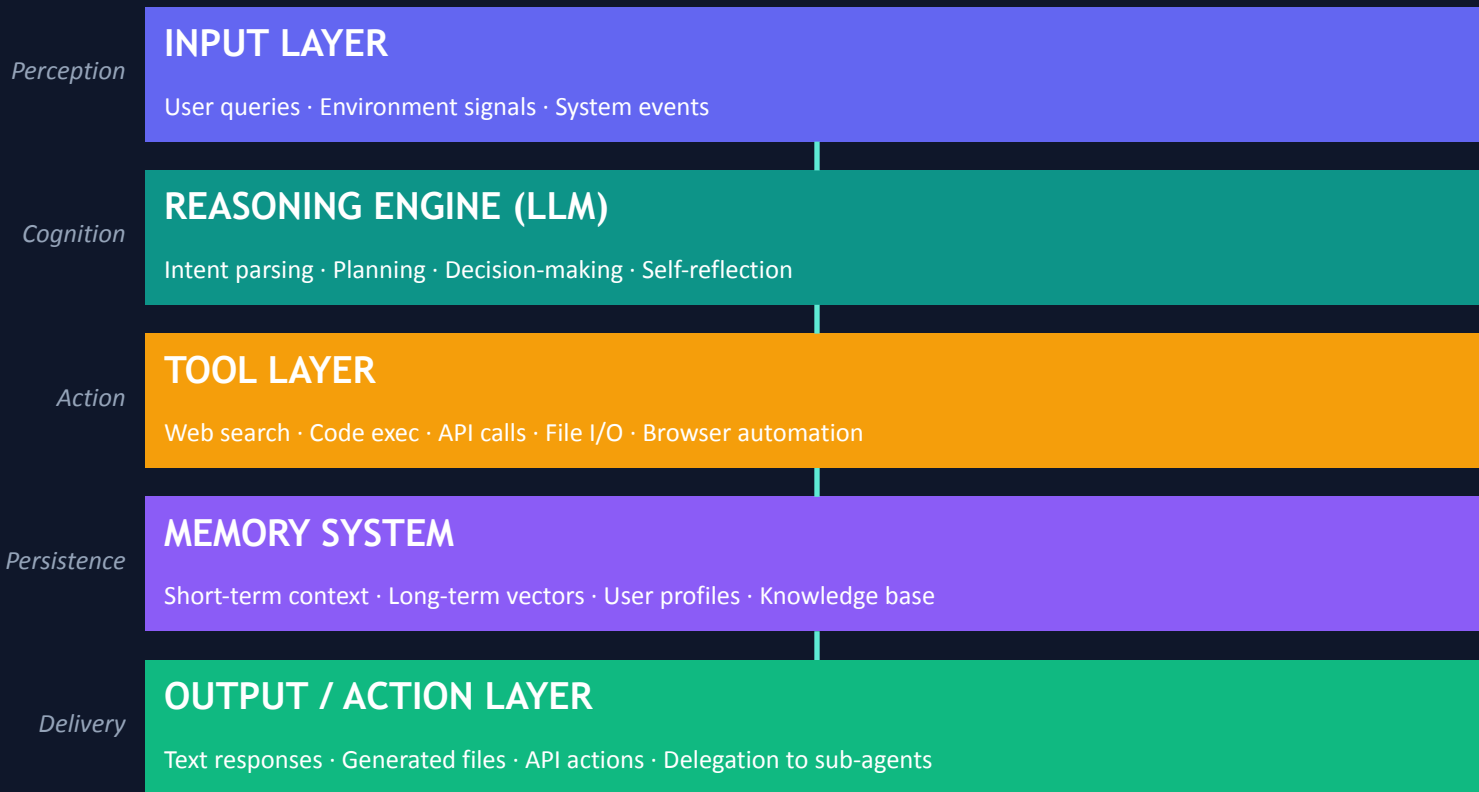
Cloud APIs, containerized sandboxes, observability tools make agents production-ready

SECTION 2

Core Architecture of an AI Agent

Components, execution loop, and design patterns

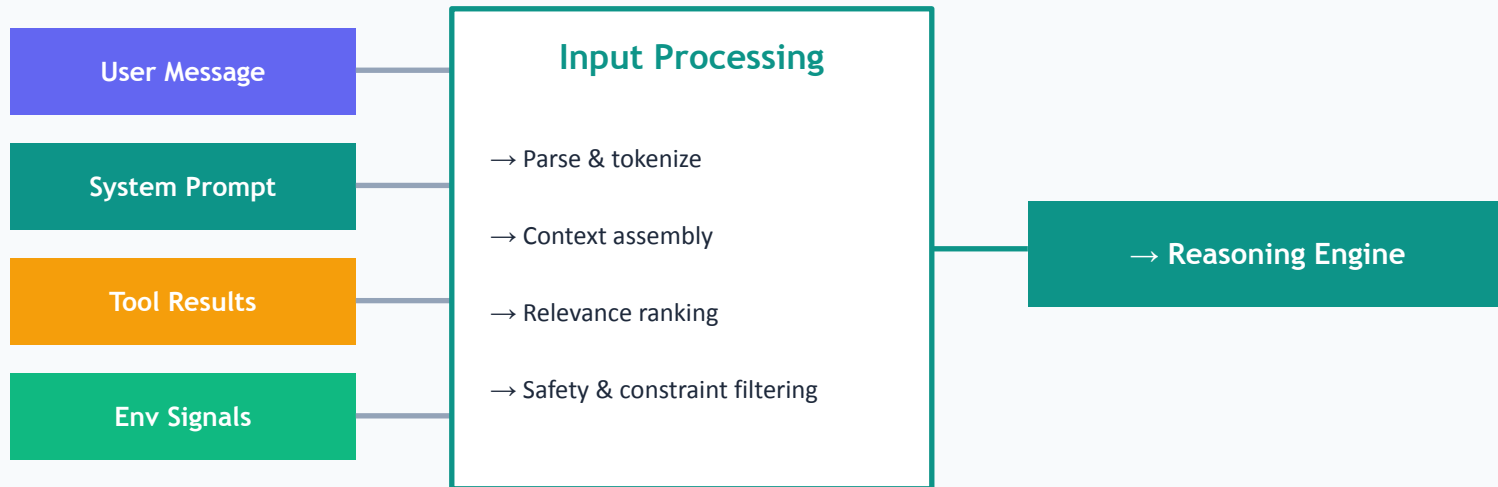
Five-Layer Agent Architecture



Input Layer



Input Sources & Processing Pipeline:



Context Window Management

Decides what fits in the LLM's context: prioritizes recent messages, relevant memory, and tool results within token limits.

Multi-Modal Input

Modern agents process text, images, PDFs, audio, and structured data: converting everything into a unified representation.

Reasoning Engine



The "brain": an LLM that interprets context, plans, decides actions, and self-evaluates.

Reasoning Pipeline:



Chain-of-Thought

Generates intermediate reasoning steps before conclusions — improves accuracy on complex tasks

Structured Output

Produces JSON function calls, tool invocations, and structured data rather than just free text

Self-Critique

Evaluates its own outputs for correctness, completeness, and alignment with the user's intent

Uncertainty Handling

Recognizes missing info and decides to search, ask the user, or flag uncertainty

Tool Layer



Tool Invocation Flow:



Information Retrieval

Web search, database queries, knowledge base lookup, document retrieval

Code Execution

Python sandbox, JavaScript runtime, SQL queries, shell commands

External API Calls

Email (SMTP), CRM (Salesforce), payments (Stripe), messaging (Slack)

File Operations

Read/write docs, parse CSVs, generate PDFs, image processing

Browser Automation

Web scraping, form filling, screenshot capture, page navigation

Agent Delegation

Sub-agent spawning, task routing, multi-agent coordination

Memory Architecture



SHORT-TERM MEMORY

Context Window	Current conversation messages
Scratchpad	Working vars, intermediate results
Task State	Current step, remaining plan
Tool Buffer	Recent tool results awaiting use



LONG-TERM MEMORY

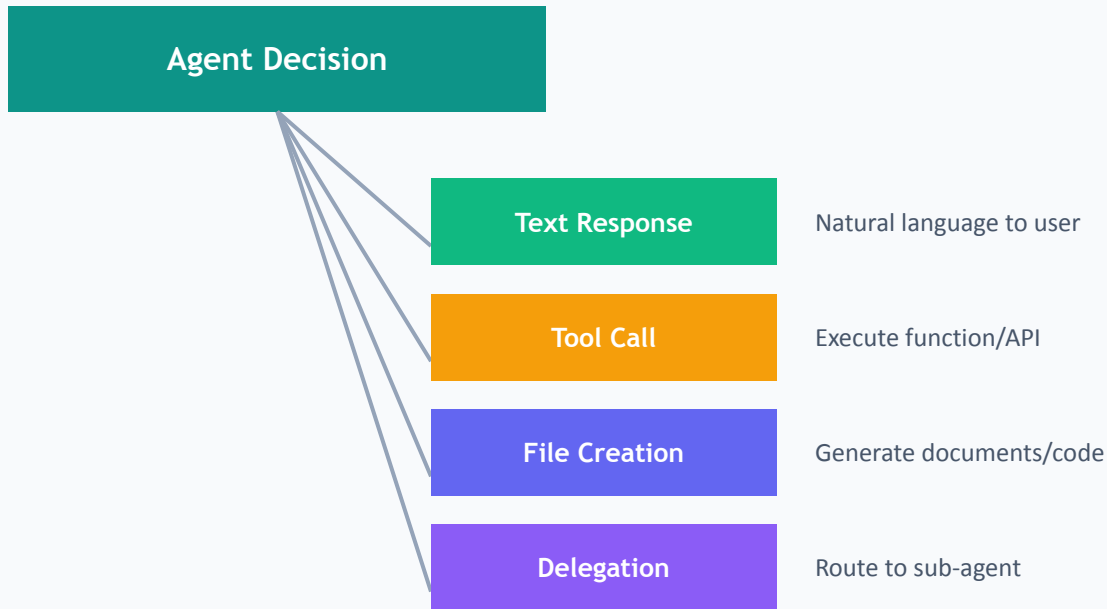
Vector Store	Embeddings of past conversations
User Profiles	Preferences, history, patterns
Knowledge Base	Domain facts, procedures, policies
Episodic Memory	Records of past outcomes

RAG: Before reasoning, the agent queries the vector store to retrieve relevant context, injecting it into the LLM's prompt alongside the user query.

Output / Action Layer



Translates the agent's decisions into concrete actions — where reasoning becomes reality.



Key Decisions:

- Should I respond or act?
- Is this the final answer?
- Do I need more info?
- Should I ask the user?
- Should I delegate?

The Agent Execution Loop



Stopping conditions: goal achieved · max iterations · user interrupt · error threshold · confidence check

Phase 1: Observe

1

Parse Trigger

Identify what initiated this iteration: user message, timer, tool result, or system event

2

Retrieve Context

Pull short-term context (conversation history) and long-term memory (vector search results)

3

Check Task State

If mid-task, load current plan, completed steps, and remaining actions from the scratchpad

4

Assess Environment

Check available tools, time constraints, rate limits, and system-level configuration

5

Identify Gaps

What information is missing? Does the agent need to search, ask the user, or make assumptions?

Phase 2: Think

1

Intent Recognition

What is the user trying to achieve? Parse ambiguity, extract structured goals, handle multi-intent queries

2

Plan Generation

Decompose the goal into ordered sub-tasks: identify dependencies and parallelizable steps

3

Tool Selection

For each sub-task, choose the optimal tool: consider accuracy, speed, cost, and availability

4

Risk Assessment

What if the API is down? Data is stale? Should I ask for confirmation before taking action?

5

Response Strategy

Stream progress, wait for completion, or ask for clarification first? Decide communication approach

Phase 3: Act

Action Execution & Result Handling:

Search: `"latest AI papers 2026"`

Returns 10 results with titles, snippets, URLs

Code: `calculate_statistics(data)`

Returns computed mean, median, std dev

API: `send_email(to, subject, body)`

Returns 200 OK confirmation

File: `create_report('analysis.pdf')`

Returns file path for download

Ask: `"Which date range?"`

Pauses loop, awaits user input

Each action result feeds back into the Observe phase for the next loop iteration.

Design Pattern: ReAct

Reason + Act: interleaves explicit reasoning with tool actions, creating a traceable, debuggable chain of thought.

Thought	I need current weather for Tokyo. Let me search.
Action	<code>web_search("Tokyo weather today")</code>
Observation	Results: Tokyo – 22°C, sunny, humidity 45%
Thought	Got the data. No rain expected → no umbrella needed.
Thought	I have enough info for a complete response.
Response	It's 22°C and sunny in Tokyo. No umbrella needed!

Key benefit: Every decision is logged and inspectable — critical for debugging, auditing, and building trust.

How Function Calling Works

The mechanism that enables LLMs to interact with external tools in a structured, reliable way.

1. Tool Definitions

Developer registers tools with name, description, and JSON Schema parameters

2. LLM Prompt

User query + tool definitions sent to model: model "sees" what tools are available

3. Model Decision

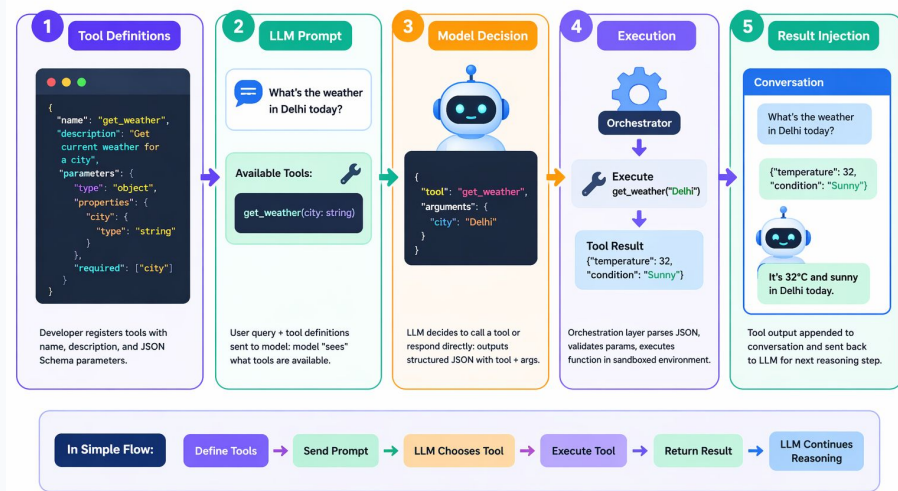
LLM decides to call a tool or respond directly: outputs structured JSON with tool + args

4. Execution

Orchestration layer parses JSON, validates params, executes function in sandboxed env

5. Result Injection

Tool output appended to conversation and sent back to LLM for next reasoning step



End-to-End Agent Execution

User: "Find the top 3 AI papers from this month and create a summary document."

Observe

Parse request → identify sub-goals: search, rank, read, summarize, create file

Think

Plan: (1) search ArXiv (2) filter by date & impact (3) read abstracts (4) generate PDF

Act #1

web_search("top AI research papers March 2026") → returns 15 results

Evaluate

Got results but need full abstracts for quality filtering → continue loop

Act #2

web_fetch(paper_urls) → retrieves 5 paper abstracts in full

Act #3

Rank papers by relevance, recency, citation count → select top 3

Think

All data gathered. Synthesize into structured summary with key findings

Act #4

create_document("AI_Summary.pdf") → generates formatted report with citations

Stop

Goal achieved → deliver file to user with brief overview of findings



SECTION 3

Types of Agents

Classifying agents by behavior, autonomy, and scale

Reactive Agents

Respond directly to stimuli without planning or state. Condition-action rules: if X, do Y.
Fast and predictable, but limited.

Reactive Agent System:



No memory · No loops · No planning

Spam filter

Keywords → mark spam

Thermostat

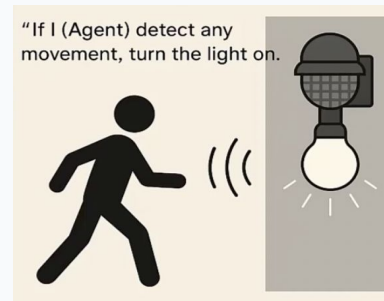
Temp > 74°F → turn AC

Auto-responder

Email → template reply

Rule chatbot

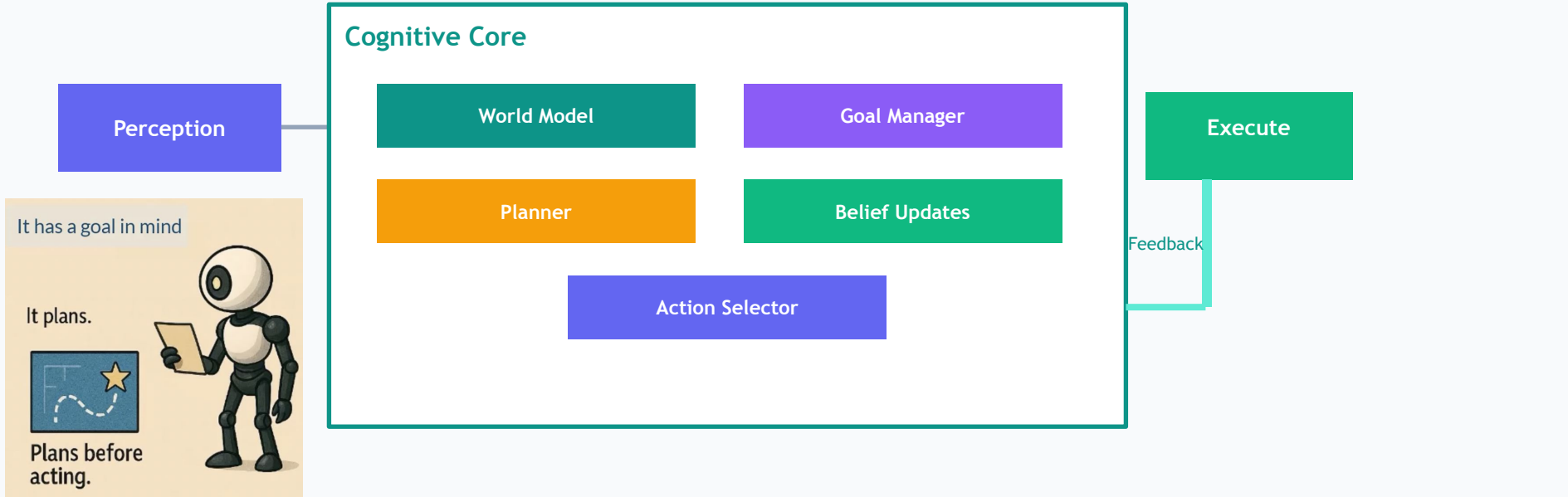
Pattern → canned response



Goal-Driven (Deliberative) Agents

Maintain a world model, formulate goals, create plans, and adapt. Reason about the future before acting, the foundation of modern AI agents.

Deliberative Agent Architecture:



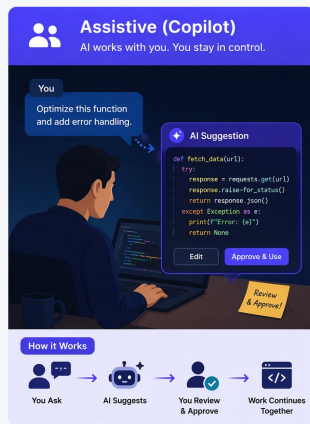
Reactive vs Deliberative: Comparison

Dimension	Reactive Agent	Deliberative Agent
Internal State	None: stateless	Full world model
Decision Speed	Milliseconds	Seconds to minutes
Planning Horizon	Current stimulus only	Multi-step future
Adaptability	Fixed rules	Dynamic replanning
Error Recovery	No retry logic	Detects & adapts
Complexity	Simple tasks	Complex, ambiguous problems
Resource Usage	Very low	Higher (LLM per step)
Example	Email auto-filter	AI research assistant

Autonomous vs Assistive Agents

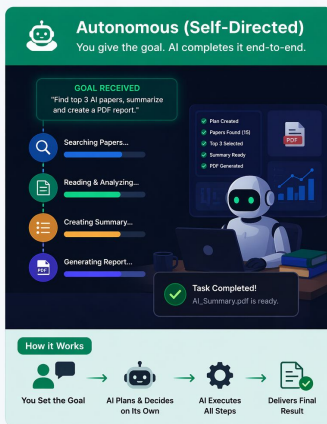
Assistive (Copilot)

- Human stays in the decision loop
- Agent suggests, human approves/edits
- Lower risk: errors caught early
- Best: creative work, code review, drafting
- Examples: GitHub Copilot, Claude chat



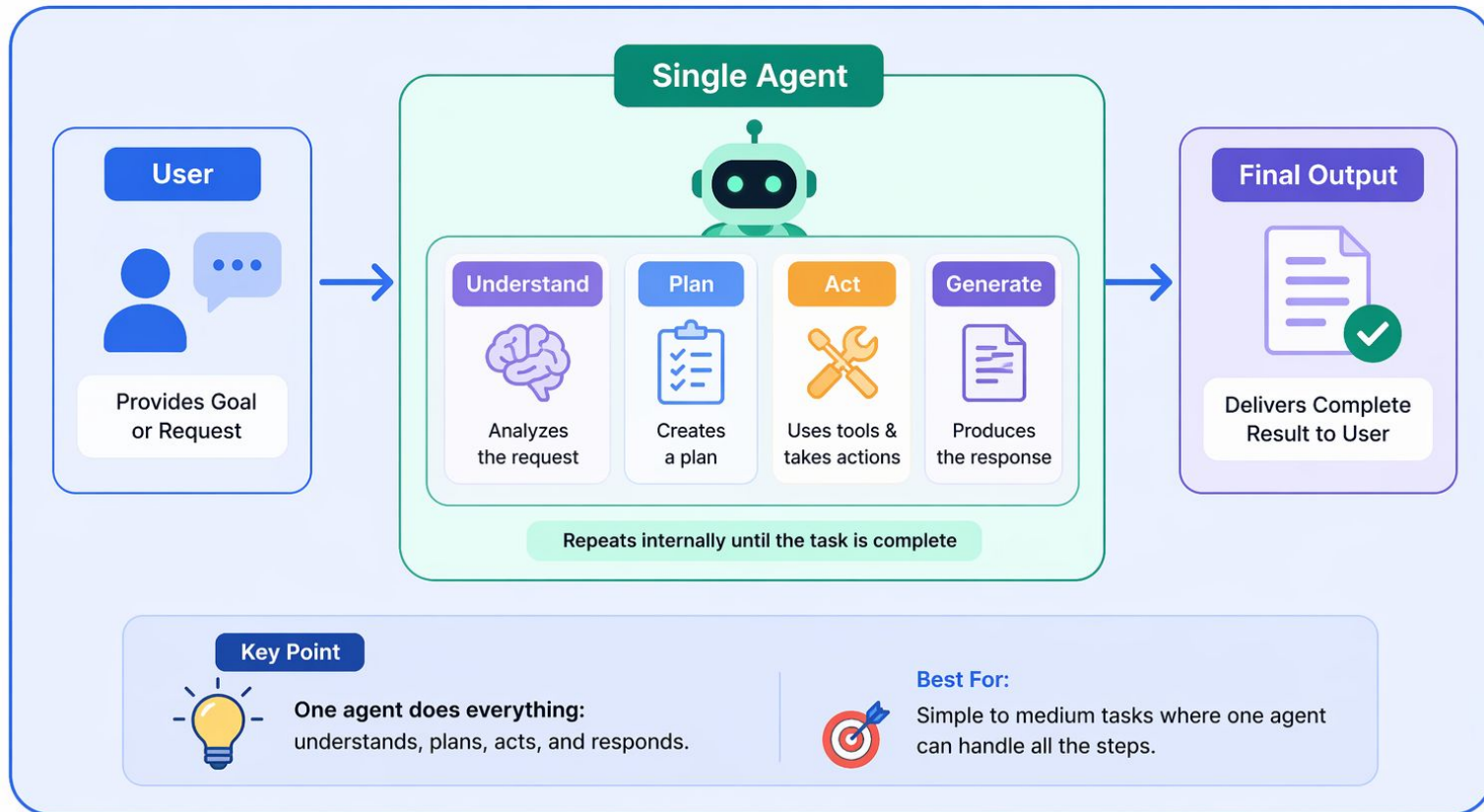
Autonomous (Self-Directed)

- Operates end-to-end without checkpoints
- Makes all decisions independently
- Higher capability but higher risk
- Best: bounded tasks, clear success criteria
- Examples: Devin, Auto-GPT, trading bots



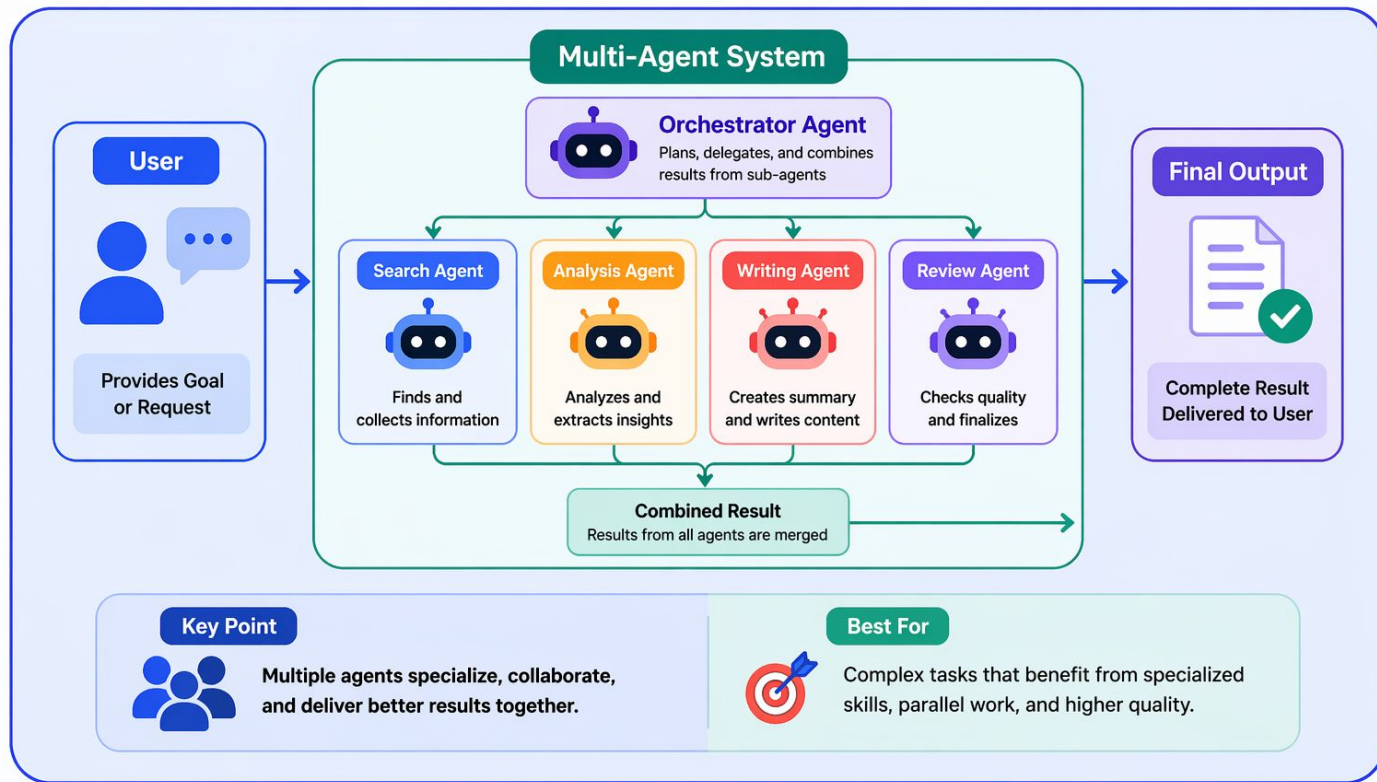
Single Agent Architecture

One agent handles the entire task from start to finish.



Multi-Agent Architecture

Multiple specialized agents collaborate to solve complex problems exceeding any single agent's capability.



Multi-Agent Communication Patterns

Hub & Spoke

Central orchestrator delegates to specialists and aggregates results. Most common — easy to manage.

Pipeline / Chain

Agents pass work sequentially — each transforms output for the next. Great for staged workflows.

Debate / Critique

Agents argue opposing positions to refine quality. Used for verification and accuracy improvement.

Swarm / Parallel

Agents work independently on sub-problems in parallel, results are merged. Maximizes throughput.

Single vs Multi-Agent Trade-offs

Factor	Single Agent	Multi-Agent
Complexity	Low: one loop, one context	High: routing, messaging, state sync
Specialization	Generalist (one model does all)	Expert per role (domain-tuned)
Scalability	Limited by context window	Distributed across agents
Debugging	Linear trace, easy to inspect	Complex interactions, harder
Cost	1 LLM call per step	N calls (one per agent per step)
Latency	Lower (single model)	Higher (coordination overhead)
Error Isolation	Single point of failure	Failures isolated per agent
Best Use	Focused, well-scoped tasks	Complex, multi-domain workflows

Agent Safety & Guardrails



As agents gain autonomy, safety becomes critical. Production systems need multiple protection layers.

Input Validation

Filter malicious prompts, injection attacks, out-of-scope requests

Action Boundaries

Allowlist of permitted tools, rate limits, resource caps per action

Approval Gates

Human-in-the-loop checkpoints for high-risk or irreversible actions

Output Filtering

Check responses for harmful content, PII leakage, policy violations

Monitoring & Logging

Full audit trail of every reasoning step, tool call, and decision

SECTION 4

Real-world Applications

From copilots to fully autonomous systems in production

AI Copilots in Production



Augment human capabilities with real-time suggestions, automated drafts, and contextual intelligence.

GitHub Copilot

Development

Code completion, test gen, refactoring, PR reviews

Microsoft 365 Copilot

Productivity

Email drafts, meeting summaries, slide creation

Cursor / Windsurf

AI-Native IDEs

Multi-file editing, codebase-aware suggestions

Claude Artifacts

Content Creation

Interactive content, code exec, data visualization

Autonomous Research Assistants



Research Agent Workflow:



Claude Deep Research

Multi-step web research with iterative search, analysis, and comprehensive report generation

Perplexity Pro

Real-time search engine with cited, structured answers and follow-up capabilities

Elicit

Academic paper discovery, data extraction, and systematic review automation

Custom RAG Agents

Domain-specific research over proprietary knowledge bases with enterprise data

Agentic AI Across Industries

Healthcare

Patient triage, clinical docs, drug interactions, diagnostic support

Finance

Fraud detection, portfolio rebalancing, regulatory compliance, risk assessment

Legal

Contract analysis, case research, document review, clause extraction

Education

Personalized tutoring, adaptive curriculum, automated grading, analytics

E-commerce

Product recs, dynamic pricing, inventory optimization, returns processing

Manufacturing

Predictive maintenance, quality control, supply chain, safety monitoring

Key Takeaways

1 Agentic AI goes far beyond chatbots: agents reason, plan, act, and iterate autonomously

2 Five-layer architecture: Input → Reasoning → Tools → Memory → Output

3 The Observe → Think → Act → Repeat loop is the universal agent execution pattern

4 ReAct creates traceable, debuggable chains of thought and action

5 Agents range from reactive to fully autonomous — choose for your risk profile

6 Multi-agent systems enable complex workflows through specialization

7 Safety guardrails are non-negotiable: validation, boundaries, oversight, monitoring

8 Real applications in production: copilots, research assistants, workflow automation



Thank You!!!

